

ROADASSIST BHARAT

What To Say

The words, in order. Everything in quotation marks is meant to be said out loud.

The one rule. Say which half is AI *before* anyone asks. Volunteering the limit in your first minute removes the only attack that really hurts, and everything after it is believed.

Opening — 30 seconds

Before you touch anything

Do: Stand still. Do not open the laptop yet.

“RoadAssist Bharat is a roadside-assistance platform for India, built for the roads where coverage is worst — highways, rural routes, disaster zones.”

“It works three ways: a smartphone app, a web app, and a feature phone over plain SMS. And it keeps working with no network at all.”

“One thing I want to say before the demo, because it matters. The road-damage detection is a genuinely trained YOLO model. The roadside diagnosis is not — that is a deterministic rule table, and every screen that shows it says so. I will show you both, and I will show you how the database tells them apart.”

If they interrupt here with “so it is not really AI” — say: “Half of it really is. Let me show you the half that is.” Then go to step 1.

The demo — five moves, about eight minutes

1 · The authority dashboard

Do: Open <http://localhost:4000/raksha.html>, sign in with +919999900001.

“This is RAKSHA — the authority side. Seventy-one road defects along the NH-48 corridor between Gurgaon and Manesar.”

“Sixty-five of these came out of our trained model. Six are simulated. The database stores the model version on every single detection, so if you want to know which is which, it is a query, not a promise from me.”

“The corridor scores underneath are a rule-based score we invented. The legend says so — it is not an official government standard, and we do not pretend it is.”

Point at the “SIMULATED DATA” badge and the legend note yourself. Being the one who points at your own caveats is the whole game.

2 · Make it do something

Do: Click **Verify** on any detection row.

“Watch the status. DETECTED becomes VERIFIED, and the button changes to Close — repaired and confirmed.”

“That is a real state machine with real transitions, not a picture of one. An illegal transition is refused by the API with a list of what actually is allowed from that state.”

3 · The citizen in trouble

Do: Switch to the emulator. Press and **hold** the SOS button.

“Notice I had to hold it, not tap it. That is deliberate — a phone in a pocket must never be able to call a responder.”

“And there is a rule underneath that I think is the most important decision in the project: a model is never allowed to dispatch. If the crash detector fires, it raises an incident that waits for a human to confirm. A false positive that sends an ambulance is worse than one that asks a question.”

“That SOS just arrived at the server with GPS to six decimal places.”

Then switch back to the RAKSHA map: “And it lands on the same corridor the authority dashboard is monitoring. Citizen side and government side are one system.” This is your strongest single moment — do not rush it.

4 · The phone that costs a thousand rupees

Do: In the terminal, send the SMS command.

“No app. No internet. A feature phone texting the word SOS with two numbers.”

“Same emergency, same database row, same coordinates. Ninety per cent of the people who most need this do not have a smartphone, so the feature phone is not a fallback for us. It is a first-class client.”

“It works in eight languages. I should be honest about that though — the strings exist and each one is tested to fit an SMS segment, but they are not native-reviewed. The count is real. The quality is unverified. Those are different claims.”

5 · The part I am proudest of

Do: Run `npm run verify` in the terminal.

"This is the bit I would like you to remember."

"Every number in our documentation — the test counts, the table counts, the route counts — is checked by the build. If a document says something the code does not support, the build fails."

"We did that because the same wrong number got into twenty documents three separate times, and a person caught it each time. A person catching it is not a mechanism."

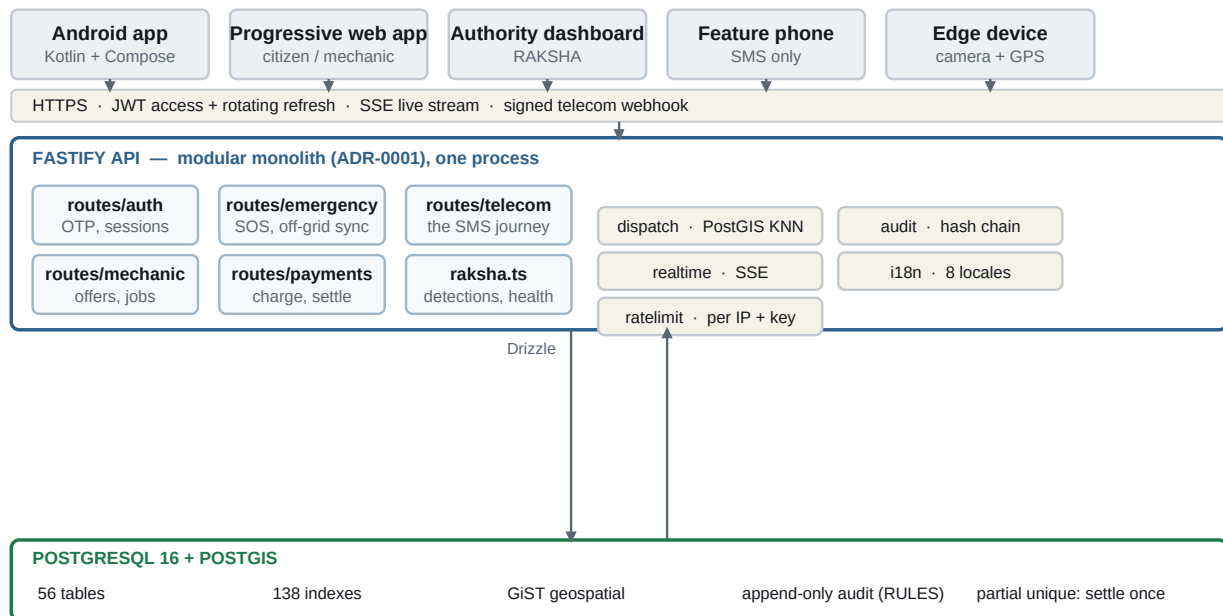
"It also checks every code reference in our viva pack still points at real code. So if you want to test any claim I have made, open the file. The line numbers still resolve, because we check those too."

If they ask you to explain the architecture

Six diagrams, each with the line to say while it is on screen. You do not need all of them — pick the one that answers what was actually asked.

A · The whole system

CLIENTS



“Five kinds of client, one API, one database. The API is a modular monolith — that was a deliberate decision, not a shortcut.”

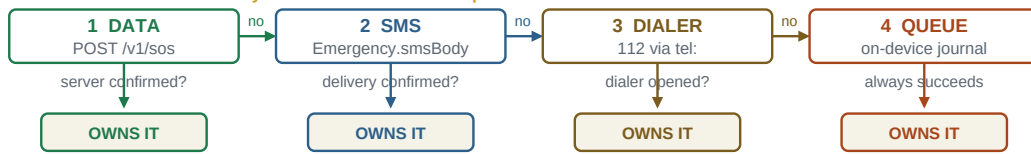
“At our size, microservices would have bought us distributed transactions and a tracing problem, and we would have spent the term on plumbing instead of the emergency path.”

“The module boundaries are still enforced though — a CI job fails the build if one module imports something it is not allowed to. So we get the discipline without the network hops.”

If asked “why not microservices” — that second line is the whole answer. ADR-0001.

B · What happens when you press SOS

THE SOS LADDER — exactly one channel owns the report



A confirmed SMS must NOT also queue an API replay. An unconfirmed one MUST.

Get it wrong and you dispatch two responders to one emergency, or none. 21 unit tests over pure functions in SosLadder.kt.

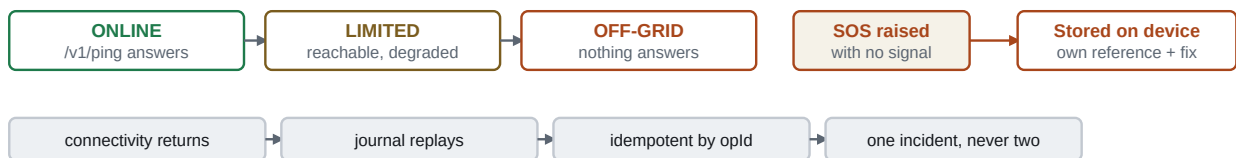
“Four rungs, and it walks down them until one works: data, then SMS, then the 112 dialer, then an on-device queue.”

“The invariant is the interesting part — exactly one channel owns the report. A confirmed SMS must not also queue an API replay. An unconfirmed one must.”

“Get that wrong and you either send two responders to one accident, or none. So those decisions live in pure functions with twenty-one tests, separate from the Android code that calls them.”

C · Working with no network

OFF-GRID MODE (ADR-0009) — three tiers the client can tell apart



The app says “stored on this device” with a reference, or it says nothing. It never claims the cloud has it.

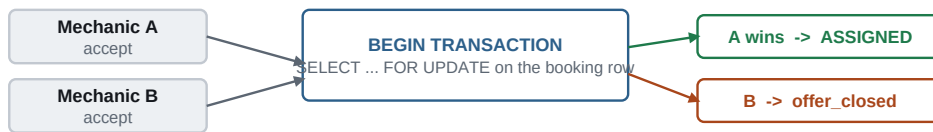
“The client tells three states apart: online, limited, and off-grid. That distinction is the product, really.”

“If you press SOS with no signal, it becomes a real incident stored on the phone, with its own reference and a GPS fix — and the app says “stored on this device”. It never tells you help is coming when it is not.”

“When signal returns the journal replays itself, and it is idempotent, so a retry can never become a second ambulance.”

D · Two mechanics, one job

TWO MECHANICS, ONE JOB — settled by Postgres, not by timing



Expiry is re-checked INSIDE the lock. The loser gets a refusal that names why, not a 500.

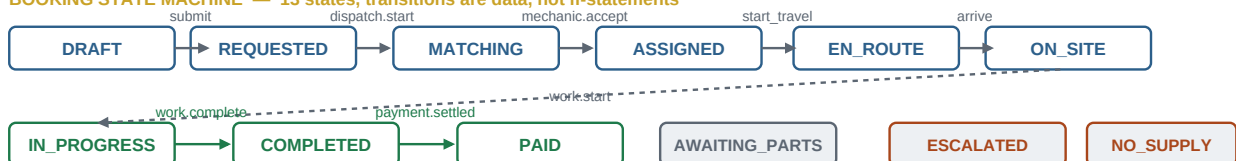
“Both accept within the same second. One wins, and it is the database that decides, not luck.”

“We take a row lock inside the transaction and re-check the offer expiry under that lock. The loser gets a refusal that names the reason, not a 500.”

“Anything that is only true because two things did not happen at the same instant is not actually true. So we push those guarantees into Postgres.”

E · The booking lifecycle

BOOKING STATE MACHINE — 13 states, transitions are data, not if-statements



An illegal command is refused with the list of what IS allowed there — the API never silently ignores a transition.

“Thirteen states, and the transitions are data rather than if-statements scattered through the code.”

“That means an illegal command is refused with the list of what actually is allowed from that state — the API never silently ignores a transition it does not like.”

F · Where RAKSHA gets its data

RAKSHA — edge to authority, and what is real at each step



65 of the 71 detections on screen are real model output. The database records model_version, so "which half is real" is a query, not a promise.

"Camera frame, model, severity, location, sync, dashboard."

"I want to be precise about which parts are real. The detections are real model output. The severity is a heuristic we wrote. And the GPS is simulated — the training images have no location data, so we place them along the corridor and label them as simulated everywhere they appear."

"Sixty-five of the seventy-one on screen are real. The database stores the model version, so you can check that yourself rather than take my word."

Questions — answer in one breath, then stop

Short answers sound confident. Long answers sound nervous. Say the sentence and stop talking.

If they ask	Say this
"Is the AI real?"	Half of it. The road-damage model is genuinely trained — best mAP50 is 0.471. The roadside diagnosis is a rule table, and it is labelled as one on every screen.
"That accuracy is low."	It is, and I will not dress it up. Potholes are the hard class at about 0.24 — small objects, and the loss was still falling when our time cap hit. More epochs on a GPU is the single biggest lever left.
"Two mechanics accept at the same second?"	One wins. We take a row lock on the booking inside the transaction and re-check expiry under that lock. There is a test suite that races them on purpose.
"Could someone be charged twice?"	No. Settlement is recorded, not asserted — there is a unique index that makes a second settled row impossible, so replaying a confirmation changes nothing.
"What if the database goes down?"	Health returns 503 naming the database, but ping still returns 200 — so the phone can tell "you have no signal" from "the platform is unwell". Our CI kills Postgres under a running server and asserts exactly that.
"Can you edit the audit log?"	No. It is hash-chained and Postgres rules make update and delete do nothing. We re-verify the chain after a backup restore in CI.
"Where is the cloud / autoscaling?"	Not built, and our slide says DESIGN rather than DONE. What is real is the readiness gate I just showed you.
"How many tests?"	636 assertions across six suites, zero failures, run this morning. There is a seventh suite of 22 payment checks that needs a Razorpay sandbox account — it did not run, so I do not count it.
"What would you do next?"	Train the model longer on a GPU, get the translations native-reviewed, and run a real load test. Those are the three things I currently cannot claim.

Never say these

Do not say	Because
"The AI diagnoses the fault."	It is a rule table. Say "rules engine".
"626 tests" or "631 tests"	It is 636 . Quote what ran today.
"The payment tests pass."	They did not run. No sandbox account.
"We tested it at scale."	No load test exists. Single user, one machine.
"Translations are done."	They exist. They are not native-reviewed.
"We support iOS / IVR / satellite."	Designed, not built. Say "roadmap".

Closing line

When they say thank you

Do: Stop clicking. Look at them.

"Everything I showed you is running on this machine right now, not on slides."

"And every number in our documents is checked by the build, so if you want to test any claim I made, open the file. That is the part we are most proud of — not that it works, but that we cannot quietly lie about whether it works."

If something breaks mid-demo, say this and keep moving: "That is my demo data, not the platform — one moment." Dashboard empty: `npm run demo:raksha`. No mechanic found: `npm run demo:reset`. App back at sign-in: sign in again, tokens are in memory by design. Never say "it usually works".